Tetris: Tile-level Sampling for Efficient and High-Fidelity Video Object Tracking

Chanwut Kittivorawong
chanwutk@berkeley.edu
U. of California, Berkeley
Berkeley, California, USA

Alena Chao
alenachao@berkeley.edu
U. of California, Berkeley
Berkeley, California, USA

Charlie Si
charliesi@berkeley.edu
U. of California, Berkeley
Berkeley, California, USA

Alvin Cheung
akcheung@cs.berkeley.edu
U. of California, Berkeley
Berkeley, California, USA

Abstract

Track materialization converts raw videos into reusable object tracks that downstream queries can run against without rerunning tracking, but extracting those tracks efficiently and with high fidelity remains expensive. Prior systems reduce track materialization cost through temporal frame sampling, but aggressive sampling spaces each track's detection points too far apart to faithfully capture the object's actual trajectory. In stationary video, however, large portions of each frame contain no objects of interest, and different sampling rates can be used to extract tracks from the remaining regions. Leveraging this idea, we present Tetris, a track-extraction system that decomposes videos into a tile-based *polyomino* data model, enabling fine-grained spatiotemporal pruning that reduces detector calls with minimal fidelity loss. Tetris implements track materialization in three steps: first, a classifier identifies relevant tiles and groups them into polyominoes. Then, we use an integer linear program (ILP) to prune redundant polyominoes under a user-specified accuracy constraint, before packing the remaining polyominoes into canvases to minimize detector calls. Across 7 stationary-video datasets, Tetris stays within a 5% tracking accuracy loss as compared to a reference pipeline that processes every frame in its entirety, while prior systems exceed this bound on 3 of the 7 datasets. Moreover, with this 5% bound, Tetris achieves up to 17.4× higher throughput than prior systems, and up to 67.5× higher than the reference pipeline. Conversely, Tetris delivers up to 0.42 higher HOTA tracking accuracy than the best prior system at matched throughput.

1 Introduction

Track materialization converts raw videos into a reusable set of object tracks that downstream analytics can query repeatedly [4, 18]. For stationary cameras deployed in traffic monitoring and surveillance, each track is a per-object trajectory that can be used in queries such as counting, retrieval, and motion analysis. Because each materialized track can serve many downstream queries, systems aim to compute tracks only once, then store and reuse them.

The bottleneck in this workload is track extraction, the compute-intensive process that produces reusable tracks from raw video. Modern tracking-by-detection pipelines spend 99.4–99.9% of their runtime inside the object detector, and long continuous video streams amplify this cost at scale. Reducing the number of detector invocations, or the amount of pixel data each invocation must process, is therefore the primary lever for improving throughput.

Many downstream tasks require fine-grained trajectories, not merely object presence. For instance, decentralized vehicle coordination in understructured traffic scenes relies on per-vehicle trajectories recovered from object tracks [35], and traffic-flow studies such as the I-24 MOTION phantom-jam experiment depend on recovering when each vehicle decelerates, re-accelerates, and whether its motion propagates or damps a stop-and-go wave [26]. Missing, imprecise, or temporally coarse tracks are harmful during abrupt motion changes; producing tracks at the fidelity that such queries require is exactly what makes track extraction expensive.

Prior track-materialization systems reduce cost through two strategies, both incurring accuracy losses. OTIF [4] and LEAP [37] use whole-frame sampling; with aggressive frame sampling, the resulting tracks contain detection points spaced too far apart to faithfully capture fine-grained motion such as abrupt decelerations. OTIF additionally crops the sampled frames with axis-aligned rectangular regions of interest (ROIs), that is, rectangles whose edges are parallel to the image axes. Such windows can still enclose irrelevant pixels when the underlying relevant region is non-rectangular. §2.2 discusses both strategies in detail.

Stationary-camera deployments are common in traffic monitoring and surveillance, and they expose a property that prior work has not fully exploited: *since the camera does not move and the scene's physical layout is fixed, the spatial distribution of relevant content is stable over time.* Objects recur in the same image regions and follow the same scene-induced motion patterns. This stability makes it possible to prune at a granularity finer than whole frames and with shapes finer than rectangular ROIs.

We identify three properties of stationary video that make this stability exploitable, to be discussed in §2.4:
Spatial irrelevance: across our 7 evaluation datasets, a mean of 94.6% of spatial regions per frame contain no objects of interest.
Spatially varying sampling: different sampling rates can be used for different regions, depending on their motion patterns, making any single whole-frame sampling rate either wasteful or inaccurate.
Tight non-rectangular clustering: relevant regions form connected but non-rectangular shapes whose axis-aligned enclosing rectangles contain on average 39% more area than the regions themselves, measured across our evaluation datasets.

To exploit these properties, we present Tetris, a track-extraction system that decomposes stationary-camera video into a tile-based *polyomino* data model [11], enabling fine-grained spatiotemporal pruning that reduces detector calls with minimal fidelity loss. Tetris divides each frame into a grid of square tiles and groups connected relevant tiles into polyominoes, non-rectangular regions that tightly enclose relevant content (§4). Importantly, Tetris is agnostic to the object detector and tracker, and it lets the user define their accuracy target (§6.3). Based on the target, Tetris's execution engine (§5) runs an optimized *tracking pipeline* of three operators (classify, prune, pack) upstream of the user-provided object detector, then unpacks the resulting detections for the tracker (§3). This extraction-time

pipeline is preceded by a one-time training phase that, once per dataset, trains a lightweight relevance classifier specialized to the user-provided detector (§6.1) and learns the maximum tile sampling gaps the pruner relies on (§6.2). Here, a dataset is a large collection of videos from a single stationary camera. These learned artifacts are specific to the dataset and the user-provided detector and tracker; changing the dataset, detector, or tracker requires new reference outputs and the affected training, profiling, and validation steps.

To use Tetris, users provide a stationary-camera video dataset, an object detector and a tracker, and either a throughput requirement or an accuracy requirement. Tetris then runs a one-time preparation step on samples from the dataset, producing the *learned artifacts* needed for efficient extraction (a relevance classifier and maximum tile sampling gaps) and a Pareto frontier of candidate configurations that helps the user select an operating point. After the user selects a configuration, Tetris's execution engine extracts the tracks on the remaining video data using that configuration, and the extracted tracks are returned to the user.

On 7 stationary-video datasets [4, 15, 35], Tetris keeps the loss in HOTA [22] tracking accuracy below 5% relative to the reference pipeline (§2.1), while prior systems exceed this bound on 3 of the 7 datasets. Furthermore, with this 5% bound, Tetris delivers 1.5–17.4× higher throughput than prior systems, and 6.5–67.5× higher throughput than the reference pipeline. With a 10% loss bound, Tetris achieves 1.6–24.7× higher throughput than prior systems and 10.2–71.8× higher throughput than the reference pipeline (§7).

In sum, we make the following contributions.

- A *tile-based polyomino data model and an execution engine* for efficient track extraction. The engine runs three operators upstream of the user-provided detector: a classifier identifies polyominoes of relevant tiles, an ILP prunes redundant polyominoes constrained by learned maximum tile sampling gaps, and a polyomino packing algorithm packs the remaining tiles into a minimum number of canvases to reduce object detector invocations.
- A *method for learning maximum tile sampling gaps* from empirical per-tile mistrack rates. For each mistrack-rate tolerance, Tetris selects the largest measured gap that remains within the tolerance at each tile, turning an exponential search over per-tile gap assignments into a one-dimensional sweep over the scalar tolerance. At runtime, these gaps become ILP constraints that let the execution engine prune polyominoes according to empirically measured mistrack rates.
- An evaluation on seven stationary-video datasets using an accuracy (HOTA) or throughput constraint. With a 5% HOTA-loss bound, Tetris meets the bound on all videos, while prior systems fail to do so on 3 of the 7 datasets. Compared with prior systems with the 5% bound, Tetris delivers up to 17.4× higher throughput. And, with the same throughput bound as prior systems, Tetris achieves up to 0.42 higher HOTA at 1000 FPS.

2 Background and Observations

We first discuss the tracking-by-detection pipeline that Tetris accelerates (§2.1), position our work within efficient track materialization (§2.2), and present empirical observations on stationary video that motivate the design choices of Tetris (§2.4).

2.1 Tracking by Detection

Tracking by detection is the most widely used paradigm for multi-object tracking [23]. As shown in Fig. 1, such pipeline takes an N-frame video as input and processes it in two stages. First, an object detector [20, 28, 29] runs independently on each of the N frames, producing a set of bounding boxes per frame, where each box is a tuple (x_1, y_1, x_2, y_2) specifying the pixel coordinates of a rectangular region enclosing a detected object. Let N' denote the number of detection calls. In this naive pipeline, $N' = N$ since the detector is invoked once per frame. Second, an object tracker performs *data association*; it links the bounding boxes across consecutive frames that likely correspond to the same physical object, assigning each a consistent track identifier to produce a set of tracks [5, 6, 33, 34, 39].

While the object detector utilizes expensive machine learning inference on every frame, data association can be performed using simple statistical methods such as bounding box overlap [5, 6, 39]. Hence, detection dominates the pipeline's computational cost and accounts for 99.4–99.9% of the detection and tracking compute of an unoptimized full-frame, every-frame tracking-by-detection pipeline, measured across the 7 evaluation datasets of §7. Reducing the number of detector invocations is therefore the primary strategy for improving end-to-end throughput. We call this unoptimized full-frame, every-frame pipeline built from the user-provided detector and tracker the *reference pipeline*; Tetris optimizes against it and we use it throughout as the accuracy and runtime baseline.

2.2 Track Materialization

Track materialization is the data management task of converting raw video into persistent object tracks that downstream queries can reuse, and Tetris accelerates the track-extraction step that produces these reusable tracks from stationary camera streams.

The primary cost of track materialization is producing those reusable tracks. As detection dominates runtime, prior systems reduce this cost by reducing detector computation. One common approach is temporal frame sampling. OTIF [4] samples frames uniformly to reduce execution time proportionally, and LEAP [37] introduces a predictive strategy that skips frames whose object states can be inferred from previously sampled frames. A complementary approach is spatial pruning within each sampled frame. For example, OTIF uses a proxy model to identify axis-aligned regions of interest (ROIs) that the detector then processes. Both approaches reduce detector computation. Temporal sampling reduces the number of frames passed to the detector, and spatial pruning reduces the amount of pixel content the detector must process within sampled frames. But each has a corresponding limitation. Frame sampling erases inter-frame motion that fine-grained tracking requires, and rectangular ROIs waste detector work when relevant contents are not perfectly rectangular. We evaluate track extraction quality with HOTA [22], which jointly penalizes missing detections, fragmented tracks, and identity switches, making it the appropriate fidelity metric for the fine-grained tracking we target. §2.4 quantifies the performance gap these limitations cause on stationary videos.

2.3 Specialization in Video Analytics Systems

Video analytics systems share region selection, sampling, grouping, and packing techniques, but specialize them to different tasks,

scene assumptions, and pipeline stages. Studying these techniques across tasks helps identify their strengths in different scenarios. Systems with compatible assumptions can complement one another by optimizing different pipeline components. Different degrees of task specialization let users choose the most specialized system that still supports their intended queries. We compare these choices to position Tetris's reusable track materialization.

RegenHance [32] groups selected macroblocks into connected regions resembling polyominoes, then packs their bounding boxes across frames for super-resolution. MOSAIC [10] uses motion and tracking to select rectangular regions of varying sizes, then resizes and packs regions from concurrent streams into detector inputs.

Tetris shares these grouping and packing building blocks, but targets track materialization. Our contribution combines the tile and polyomino data model with an execution engine that operates on these units for joint spatial and temporal selection of detector input. Guided by observations from stationary video (§2.4), we remove irrelevant content, group the remainder tightly, and retain regions at different frequencies. Keeping less content reduces detector work but risks missing detections or breaking associations. We learn each location's sensitivity to sampling and validate configurations to balance throughput against tracking fidelity.

In multi-device settings, systems optimize offloading and camera coordination. Glimpse [7] selects whole cached frames for local tracking and triggers server recognition; PreActo [27] predicts cross-camera handovers. In video compression, How2Compress [36] reduces encoded bytes through macroblock quantization, while Crucio [40] combines frame filtering, learned compression, and decoding optimizations. Tetris complements these coordination and codec optimizations by pruning and packing decoded content to reduce the remaining detector computation.

OTIF and LEAP remain our closest object-level materialization baselines (§2.2). NoScope [15] uses frame-difference filters to reject predicted empty frames or reuse presence labels. Seiden [2] allocates whole-frame samples with a bandit strategy guided by variation in sampled query results, then interpolates labels for unsampled frames. These frame-level labels do not by themselves provide per-object trajectories. Tetris prunes within frames even when tracking requires frequent frame sampling, supporting reuse of detailed trajectories across downstream queries.

2.4 Observations from Stationary Videos

We present empirical evidence from our seven stationary-video datasets for the three properties claimed in §1. We formally describe these datasets in §7; here, we use two of them as running examples: CalDoT1, a highway traffic camera from the OTIF benchmark [4], and B3D4, an intersection camera dataset [35].

Observation 1: Spatial Irrelevance. Fig. 2 Left shows the percentage of frames in which each tile location contains a detected object in CalDoT1. Most tile locations are rarely relevant, and many are never relevant. Across our 7 datasets from different traffic cameras, a mean of 94.6% of tiles per frame contain no objects of interest, quantifying the opportunity for tile-level filtering. A small image classifier over fixed-size tiles can identify them 26–44× faster than the detector running on the same frames (§7.4.6).

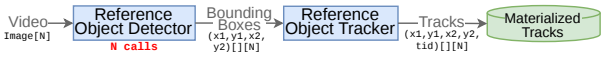


Figure 1: Tracking-by-detection pipeline.



Figure 2: Left Per-tile relevance percentage across CalDoT1. Most tile locations are rarely or never relevant. Middle Per-tile mistrack rate (%) for B3D4 when sampling every 4 frames. Right A connected group of relevant tiles (green tiles) and its axis-aligned window (red dotted lines). Red tiles are irrelevant but included in the window.

Observation 2: Spatially-Varying Sampling. Fig. 2 Middle visualizes the per-tile mistrack percentage on the B3D4 dataset when sampling every four frames. In the B3D4 intersection scene, tiles along the approaching lanes (right lanes) of each leg exhibit higher mistrack rates than tiles along the exiting lanes (left lanes). This is because vehicles approaching the intersection accelerate and decelerate unpredictably, making their positions harder for the tracker to interpolate across skipped frames. Tiles immediately before the stop line are an exception because vehicles are nearly stationary and thus easy to track. Vehicles exiting the intersection maintain steady speed, allowing the tracker to predict their positions accurately even with aggressive frame skipping. These spatial divergences in tracking difficulty are reflected in the high variance of per-tile mistrack rates. Across the seven datasets combined, the standard deviation is 20.6%. On individual datasets, the rates vary significantly as well; for example, the standard deviation is 18.2% on B3D4 and 35.5% in CalDoT1. These spatial patterns show that using a single whole-frame sampling rate is suboptimal, and that a system that learns maximum tile sampling gaps can significantly reduce the number of tiles processed with minimal tracking accuracy loss.

Observation 3: Tight Spatial Grouping. Fig. 2 Right compares the area of a connected relevant-tile group against the area of its axis-aligned window in B3D4. On average across our datasets, axis-aligned windows contain 39% more tiles than the connected groups they enclose. This overhead compounds with the pruning from Observations 1 and 2, since a system that prunes at the granularity of rectangular regions reintroduces irrelevant content that tile-level pruning would have discarded.

From observations to system design. These three observations motivate the core components of Tetris: tile-level relevance classification (§5.1, Obs. 1), per-tile polyomino pruning, which skips more tiles in regions that tolerate infrequent observation (§5.2, Obs. 2), and polyomino grouping rather than rectangular ROIs (§4, Obs. 3).

3 Tetris Overview

Consider a user with a large video dataset captured by stationary cameras and a user-provided object detector and tracker for

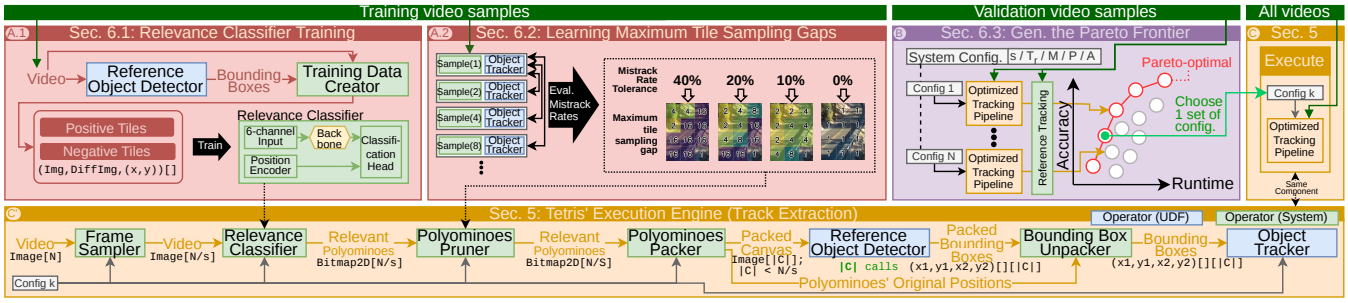


Figure 3: System overview. The learning stage produces the learned artifacts: the relevance classifier (§6.1) and maximum tile sampling gaps (§6.2). Pareto frontier generation produces a throughput-accuracy curve from swept system configurations (§6.3). The user selects a configuration, then the Tetris execution engine (§5) uses it to extract tracks on the full dataset. UDF operators are supplied by the user (the detector, and optionally the tracker); System operators are Tetris's optimization operators.

identifying and tracking objects of interest (e.g., vehicles in traffic monitoring). The user's goal is to materialize the tracks for all objects of interest across the dataset, subject to a throughput or accuracy requirement. Tetris accelerates track extraction under either requirement, maximizing throughput subject to a bound on accuracy loss or maximizing accuracy subject to a throughput floor.

To achieve this goal, Tetris first produces the *learned artifacts* (a relevance classifier and maximum tile sampling gaps) and a *throughput–accuracy frontier* that helps the user choose an operating point; only then does it extract the *final tracks* by applying the selected configuration to the remaining video data. The artifacts and the frontier guide extraction, but they are not the final tracks. Tetris does not change how tracks are represented or queried downstream; it changes how efficiently they are extracted.

At its core, Tetris divides each video frame into a grid of square *tiles* and treats tiles as the fundamental unit of processing. Each tile is classified as either relevant (containing objects of interest) or irrelevant (background), allowing Tetris to discard large portions of a frame without running the expensive object detector. Connected relevant tiles within a frame form a *polyomino*, an arbitrary two-dimensional shape composed of edge-adjacent tiles, which must be processed together to avoid splitting objects across tile boundaries. Not every polyomino needs to be processed in every frame. Our learned *maximum tile sampling gaps* determine how frequently each region must be sampled to keep tracking accuracy within a given tolerance. After pruning, the remaining polyominoes are packed into rectangular *canvases* for batch object detection. We formally define tiles and polyominoes in §4.

Shown in Fig. 3, Tetris learns the artifacts and generates the Pareto frontier on small samples, then extracts tracks on the full dataset with a user-selected configuration. Tetris samples two disjoint video subsets from the input video dataset: a training sample for learning the relevance classifier and maximum tile sampling gaps and a validation sample for generating the Pareto frontier. The sample sizes are user-settable; our evaluation uses at most 60 one-minute videos per sample (§7). Once the user chooses a configuration from the generated Pareto frontier, Tetris then extracts tracks efficiently from the remaining data via the Tetris execution engine (§5). This workflow is agnostic to the choice of object detector and tracker. Its learned artifacts and throughput–accuracy frontier are specific to each dataset and detector/tracker combination and Tetris regenerates them when that combination changes.

Table 1: System configuration parameters.

Parameter	Candidate values considered
Whole-frame sampling rate s	1, 2, 4, 8, 16 (every s-th frame; §6.3)
Relevance threshold T_r (§5.1)	0.25, 0.5, 0.75
Mistrack-rate tolerance $\bar{M}$ (§5.2)	none, 0.2, 0.4, 0.6
Tile padding P (§5.3)	0; 0.5 tile (top-right); 0.5 tile (bottom-left); 1 tile
Tracker option A (§5.4.3)	user-provided tracker, SORT

Learned Artifact Construction (§6.1, §6.2). Using the training sample, Tetris trains a lightweight relevance classifier specialized for the user-provided detector (§6.1) and learns maximum tile sampling gaps (§6.2). The user-provided detector and tracker together define the unoptimized full-frame, every-frame *reference pipeline* (Fig. 1). Because the input video lacks ground-truth tracks, Tetris uses this pipeline's tracking results as the target for accuracy evaluation. The relevance classifier learns to predict whether a tile contains objects of interest from the tile's color image, the difference between consecutive frames, and the tile's position. The maximum tile sampling gaps specify, for each tile location, how frequently that location must be sampled to maintain accurate tracks.

Pareto Frontier Generation (§6.3). Using the validation sample, Tetris runs the unoptimized reference pipeline to establish the reference tracks, then Tetris's execution engine extracts tracks from the validation sample under a range of configurations (Table 1). Each configuration produces a throughput-accuracy operating point. The output is a set of Pareto-optimal configurations that lie on the throughput-accuracy frontier, each representing a tradeoff between throughput and tracking accuracy against the reference tracks.

Track Extraction (§5). Given the user's throughput or accuracy requirement, the user selects a configuration from the Pareto frontier, and Tetris's execution engine extracts tracks on the remaining video data by executing a tracking pipeline of three Tetris operators upstream of the user-provided detector, followed by a lightweight unpacking step before the tracker. First, the relevance classifier evaluates each tile and groups connected relevant tiles into polyominoes (§5.1). Second, an ILP prunes polyominoes based on the maximum tile sampling gaps to minimize the total number of tiles processed (§5.2). Third, a two-dimensional first-fit-descending

algorithm packs the surviving polyominoes into rectangular canvases of the same resolution as the original video frames (§ 5.3). The user-provided object detector then processes each canvas, and Tetris unpacks the resulting bounding boxes back to original frame coordinates for the tracker (§ 5.4).

Specialization and scope. Tetris processes stored video on a single server without distributed camera coordination. Its learned artifacts are specific to each dataset and detector/tracker pair. Changes to the dataset, video distribution, or either model require repeating the affected learning and validation. We train, validate, and test within each stationary-camera dataset (§ 7); cross-camera transfer and robustness to weather or traffic shifts are not evaluated. Preparation amortizes over further videos from the same distribution (§ 7.2).

4 Data Model for Optimization

Tetris operates on a tile-level data model that lets its execution engine process only regions relevant to object tracking. This section defines its two core abstractions: tiles and polyominoes.

4.1 Tiles and Relevance

Each video frame is divided into a grid of square tiles of size $TS \times TS$ pixels. For a frame of height H_F and width W_F, the grid contains $H = \lfloor H_F/TS \rfloor$ rows and $W = \lfloor W_F/TS \rfloor$ columns; frames with sizes not divisible by TS are resized to fit the grid. We denote the tile at row i and column j for a particular frame by its position (i, j).

Tetris classifies each tile as either *relevant* or *irrelevant* for object tracking. Relevant tiles visually contain the objects of interest (e.g., vehicles in traffic monitoring), while irrelevant tiles contain only background content such as empty roads, trees, or sky, as shown in Fig. 4 (A). Formally, a relevance classifier assigns each tile a score $Score_{i,j} \in [0, 1]$, where values close to 1 indicate high relevance and values close to 0 indicate irrelevance. A tile is relevant if $Score_{i,j} \geq T_r$ for a threshold T_r; otherwise it is irrelevant. The threshold T_r is a system configuration parameter, set by the configuration the user selects after Pareto frontier generation (§ 6.3). Irrelevant tiles are discarded entirely, saving Tetris from running the expensive object detector on regions that contain no objects of interest. The design and training of the relevance classifier are described in § 5.1.

Tetris uses TS=60. Smaller tiles prune irrelevant content more finely but increase the number of tiles, and thus the classification time, while larger tiles retain more irrelevant background pixels.

4.2 Relevant Polyominoes

An object of interest may span multiple adjacent tiles within the same frame. If each relevant tile were processed independently by the object detector, the detector could incorrectly localize the object, for instance, reporting two half-detections for a vehicle that straddles two tiles. Therefore, connected relevant tiles must be processed together, as shown in Fig. 4 (A).

We group connected relevant tiles into a *polyomino* [11, 17], an arbitrary shape formed by edge-connected tiles on a grid. Let P denote the list of disjoint relevant polyominoes in the frame. Each polyomino $p_k \in P$ $(1 \leq k \leq |P|)$ is a set of tile positions on the grid and $|p_k|$ denotes the number of tiles in p_k. A polyomino is a unit for joint pruning and packing and may contain multiple objects. It does not represent an object segmentation mask or identity.

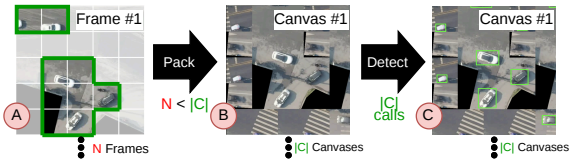


Figure 4: A video frame divided into a grid of tiles. Relevant tiles (highlighted) contain objects of interest, while irrelevant tiles (grayed) are discarded to save computation. Connected relevant tiles form a polyomino (green border). Polyominoes from multiple frames are packed into a single canvas.

Unlike prior systems such as OTIF [4] that group relevant tiles into the smallest enclosing rectangular region of interest (ROI), polyominoes can be of arbitrary shape. This flexibility is important because rectangular ROIs can be highly wasteful. For example, if objects align diagonally across a frame, the smallest enclosing rectangle may span the entire frame, forcing the system to process large irrelevant regions. Polyominoes avoid this excess coverage by tightly following the shape of the relevant area. The procedure for constructing polyominoes from classified tiles is detailed in § 5.1.

In subsequent sections, when discussing multiple frames, we add a frame index f to these symbols (e.g., $Score_{f,i,j}$, P_f, $p_{f,k}$).

5 Tetris Execution Engine

Once the user selects a system configuration (Table 1), Tetris's execution engine extracts tracks on the remaining video data. The engine consumes the two learned artifacts constructed in § 6, the relevance classifier and the maximum tile sampling gaps; this section treats them, along with the selected configuration, as given. The goal is to reduce time needed to materialize tracks while maintaining tracking accuracy. At a high level, the execution engine identifies the regions of each frame that are relevant to tracking, and packs them into canvases that reduce the number of calls to the object detector, as shown in Fig. 3 C′.

The execution engine runs an optimized tracking pipeline composed of 3 Tetris operators upstream of the user-provided detector, followed by a lightweight unpacking step before the tracker. The detector must be user-provided because it encodes domain-specific knowledge of what objects look like; the tracker only consumes per-frame bounding boxes, so the user may provide a tracker or use Tetris's built-in default, SORT [5]. Before the 3 operators run, the engine applies standard whole-frame sampling, processing only every s-th frame ($s = 1$ retains every frame). The rate s is set by the user-selected configuration (Table 1). The 3 Tetris operators are:

(1) **Relevance Classification (§5.1)** identifies the regions in video frames that contain objects of interest. A video frame is divided into a grid of square tiles. Tiles that contain objects of interest are marked as relevant. The rest are marked as irrelevant. Connected relevant tiles within the same frame form a polyomino. The operator outputs a list of relevant polyominoes.
(2) **Polyomino Pruning (§5.2)** samples at the tile level rather than frame level. Given the maximum tile sampling gaps learned offline, it prunes polyominoes whose tiles are covered by neighboring frames, minimizing the number of tiles to process.

(3) **Polyomino Packing (§5.3)** packs the remaining polyominoes into canvases to minimize the number of detector calls.

After these three operators, Tetris invokes the user-provided detector on each packed canvas, unpacks the resulting detections back to their original frame coordinates, and feeds them to the selected tracking algorithm to produce object tracks (§5.4).

5.1 Relevance Classification

Prior work [4] in video analytics systems uses coarse-grained proxy models to identify regions that contain objects of interest, avoiding calls to the detector on regions predicted to be irrelevant. Unlike these prior proxy models, which classify regions from pixel values alone, Tetris's relevance classifier additionally conditions on per-tile motion (frame differences) and spatial position, which is informative for stationary videos. Frame differences highlight motion that is usually absent from background tiles, while tile position captures stable scene priors such as lanes, sidewalks, and sky. Together, these cues help determine tile relevance in visually ambiguous regions where RGB appearance alone leaves useful evidence unused.

Model Architecture. Building on the observations above, we note two additional cues for tile-level classification: the frame-difference image is a strong indicator of relevance (moving objects produce visible local changes), and tile position provides a stable spatial prior (e.g., sky tiles are rarely relevant). Our classifier therefore takes the tile's RGB image, the frame-difference image, and the tile position as input, and determine if the tile contains objects of interest to the user-provided detector. We use ShuffleNet [24] as the backbone, and extend it to additionally take in the difference between consecutive frames and tile positions.

We now formalize the classifier's inputs, its per-tile scoring function, and its per-frame output. At video frame f, the tile position is represented as a 2D coordinate (i, j). Let $\text{Img}_{f,i,j}$ denote the RGB tile image at position (i, j) in frame f, and $\Delta\text{Img}_{f,i,j}$ the corresponding frame-difference image, defined as $\Delta\text{Img}_{f,i,j} = |\text{Img}_{f,i,j} - \text{Img}_{f-1,i,j}|$, where $|\cdot|$ denotes element-wise absolute value.

Our classification function on each tile is defined as

$$Score_{f,i,j} = \text{Relevance}(\text{Img}_{f,i,j}, \Delta\text{Img}_{f,i,j}, (i, j)) \quad (1)$$

The score is a scalar value between 0 and 1, with 1 indicating high relevance and 0 indicating irrelevance. If $Score_{f,i,j}$ is greater than a threshold T_r, the tile at position (i, j) in frame f is considered relevant. Otherwise, the tile is considered irrelevant.

For a particular frame f, Tetris outputs a matrix $Scores_f$ of size $H \times W$, where each element $Score_{f,i,j}$ is the relevance score for the tile at position (i, j) in frame f.

The architecture of our classifier is shown in Fig. 5. The first layer of the backbone of an off-the-shelf classifier is a convolutional layer that accepts the color image (3 channels). Here, we replace the first layer with a 6-channel convolutional layer that accepts the color image and the difference between consecutive frames. Accepting the image frame difference allows the model to capture the motion of the objects in the video. In the classification head, we concatenate the feature output from the backbone (visual and motion features) with the tile position embeddings, and pass the concatenated output through two fully connected layers to obtain the relevance score for each tile. The position embeddings are computed through a learned fully connected layer instead of by directly passing the tile position to the classification head. The final output is a scalar value between 0 and 1, where 1 means the tile is highly relevant, and 0 means the tile is irrelevant.

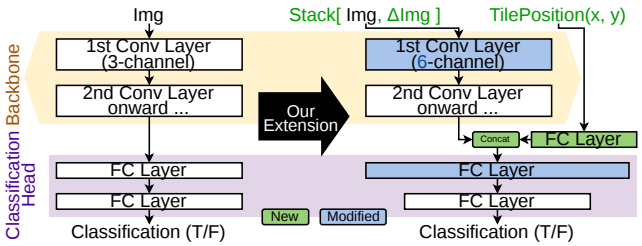


Figure 5: Left An off-the-shelf image classifier. Right Our relevance classifier architecture.

Output Polyominoes. After classifying all tiles in a frame, Tetris groups connected relevant tiles into polyominoes as defined in §4.2. The resulting polyominoes cannot be passed to the object detector directly due to their irregular shapes; §5.3 describes how Tetris packs them into rectangular canvases for batch detection.

5.2 Polyomino Pruning

Prior work in frame sampling [2–4, 15, 19, 37] either includes all relevant regions in a video frame or excludes the whole video frame from processing. This is wasteful because some regions of the video frame may require more frequent sampling to track objects accurately. For example, as observed in §2.4, tiles along approaching lanes at an intersection exhibit higher mistrack rates because vehicles accelerate and decelerate unpredictably, making their positions harder for the tracker to interpolate across skipped frames. Conversely, vehicles exiting the intersection maintain steady speed, allowing the tracker to predict their positions accurately even with aggressive frame skipping. Therefore, we develop a novel sampling method that performs tile-level rather than frame-level sampling.

5.2.1 Maximum Tile Sampling Gaps. To sample at the tile level, Tetris bounds how long each tile location may go unsampled. Given a mistrack-rate tolerance $0 \le \bar{M} \le 1$ (Table 1), Tetris derives a matrix $\bar{G}^{\bar{M}}$ of maximum tile sampling gaps, of size $H \times W$ over the tile grid, as illustrated in Fig. 3 A.2. Here, (i, j) indexes a tile location in a frame's $H \times W$ tile grid (§4.1). Each element $\bar{g}^{\bar{M}}_{i,j}$ is a positive integer that states the maximum gap between consecutive samples of tile at location (i, j) before the tracker will exceed the set mistrack tolerance rate of $\bar{M}$. Our optimization objective is hence to minimize the number of tiles to sample, while adhering to the maximum tile sampling gaps $\bar{G}^{\bar{M}}$. §6.2 describes how Tetris obtains $\bar{G}^{\bar{M}}$ based on the per-tile mistrack rates measured empirically.

5.2.2 Optimization Problem. Sampling tiles is not trivial because we cannot simply consider each tile to be independent of the others. As explained in §5.1, connected relevant tiles must be processed together; they cannot be split apart. If a tile is sampled, the whole polyomino that contains the tile must be sampled. This problem then becomes choosing a subset of relevant polyominoes to process, such that the total number of tiles to sample is minimized,

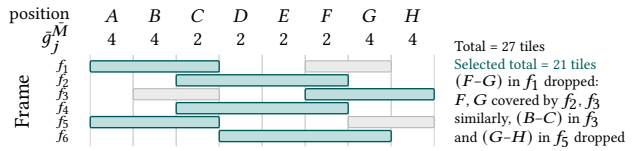


Figure 6: A 1D illustration of polyomino pruning with heterogeneous maximum tile sampling gaps. Each row shows one frame's polyominoes; teal boxes are kept and gray boxes are pruned. As a result of pruning, 21 out of 27 tiles are kept.

while strictly adhering to the maximum tile sampling gaps $\bar{G}^{\bar{M}}$. Fig. 6 shows a simple 1D example. Although the actual problem is two-dimensional, the 1D abstraction makes the coupling between per-position maximum tile sampling gaps and polyomino-level selection easier to see. In the example, each frame contains one or two polyominoes of varying shapes, and per-tile maximum tile sampling gaps range from 2 to 4. The solver exploits the fact that some polyominoes' positions are already covered by neighboring frames: for instance, positions F and G in frame f_1 are covered by the polyominoes kept in f_2 and f_3, so the solver drops the polyomino $(F–G)$ while retaining the polyomino $(A–B–C)$. This partial-frame pruning reduces the total from 27 tiles (frame-level selection) to 21, a saving that frame-level approaches cannot achieve.

5.2.3 ILP Formulation. In general, solving this problem is NP-hard (proof in § A.1). Therefore, we formulate it as an integer linear program and rely on a modern ILP solver, which handles Tetris's instance sizes efficiently in practice. Specifically, assume that we select the subset of relevant polyominoes from N video frames.

- Let P be the set of all relevant polyominoes in all N frames.
- Let P_f be the list of relevant polyominoes in frame f.
- Let $p_{f,k}$ be the k-th relevant polyomino in frame f.
- Let $s_{f,k}$ be a binary variable that is 1 if the polyomino $p_{f,k}$ is selected, and 0 otherwise.
- Let $Size(p_{f,k})$ be the number of tiles in the polyomino $p_{f,k}$.
- Let $Covers(p_{f,k}, i, j)$ be a Boolean function that returns *true* if the polyomino $p_{f,k}$ covers the tile at position (i, j). Formally,

$$Covers(p_{f,k}, i, j) \coloneqq \begin{cases} \text{True} & \text{if } (i,j) \in p_{f,k}, \\ \text{False} & \text{otherwise.} \end{cases}$$

- Let $CoverageSpan(f_{st}, f_{en}, i, j)$ be the set of binary variables $s_{f,k}$ for corresponding polyominoes that cover tile location (i, j) in frames f_{st} through f_{en}. Formally,

$$CoverageSpan(f_{st}, f_{en}, i, j) \coloneqq \left\{ s_{f,k} \;\middle|\; \begin{array}{l} f_{st} \le f \le f_{en}, 1 \le k \le |P_f|, \\ Covers(p_{f,k}, i, j) \end{array} \right\}.$$

As a shorthand, we define the coverage span with size $\bar{g}_{i,j}^{\bar{M}}$ starting at frame f and tile at position (i, j) as

$$CS(\bar{M}, f, i, j) \coloneqq CoverageSpan(f, f + \bar{g}_{i,j}^{\bar{M}} - 1, i, j).$$

We can then represent the problem as a set of constraints as follows. For each tile at position (i, j), each coverage span of size $\bar{g}_{i,j}^{\bar{M}}$ must either contain at least one selected polyomino or be empty:

$$\forall i \forall j \forall f \in [1,\, N - \bar{g}_{i,j}^{\bar{M}} + 1] : \quad (1 \le \sum_{s \in CS(\bar{M},f,i,j)} s) \;\vee\; (CS(\bar{M}, f, i, j) = \emptyset)$$

This enforces that each tile position (i, j) has a sampling gap no greater than $\bar{g}_{i,j}^{\bar{M}}$ frames. The quantification ranges only over windows that fit within the N-frame horizon. A window extending past frame N imposes no constraint, since fewer than $\bar{g}_{i,j}^{\bar{M}}$ frames remain. However, empty spans are permitted for tiles that are not relevant for $\bar{g}_{i,j}^{\bar{M}}$ consecutive frames, preventing infeasible ILPs.

Given the constraints, the objective of the ILP problem is to minimize the number of selected tiles, which is:

$$\sum_{f=1}^{N} \sum_{k=1}^{|P_f|} s_{f,k} \times Size(p_{f,k})$$

The ILP outputs a set of selected polyominoes P^s, defined as

$$P^s \coloneqq \left\{ p_{f,k} \mid s_{f,k} = 1 \right\}.$$

The objective minimizes the number of selected tiles while satisfying the maximum tile sampling gaps. Tetris limits solving to 0.1 s and uses its best feasible solution, which need not be optimal.

To keep the ILP size independent of video length, Tetris solves a separate ILP for each 1-minute window of video. A longer video adds windows, so pruning cost grows linearly with video length.

5.3 Polyomino Packing

After pruning, the surviving polyominoes must be converted into detector-compatible inputs. Our key idea is to pack multiple polyominoes, potentially from different frames, into fixed-size 2D canvases. Each canvas has the same resolution as the original video frame, so one detector invocation can process several relevant regions at once. This reduces the number of detector calls without changing the detector itself. Keeping the original frame resolution also preserves object scale, avoiding the scale mismatch that rescaling would introduce for the user-provided detector.

5.3.1 Optimization Problem. Given a set of relevant polyominoes $P = \{p_1, p_2, \ldots, p_n\}$, we aim to pack them into 2D canvases of size $H \times W$ tiles, equivalent to the original frame size of $H_F \times W_F$ pixels. The packed polyominoes must not overlap with each other. The optimization objective is to pack all the polyominoes into the fewest canvases possible without any polyominoes overlapping.

5.3.2 FFD Polyomino Packing Algorithm. Solving the packing problem is NP-hard (proof in § A.2). To obtain an approximate solution, we designed an algorithm inspired by the first-fit-descending (FFD) bin-packing [1]. The algorithm sorts polyominoes by descending size and places each polyomino into the first existing canvas where it fits without overlap. If no existing canvas can fit it, the algorithm opens a new canvas. Unlike one-dimensional bin packing, packing feasibility depends on both the remaining free area and the geometric placement of the polyomino. This heuristic does not guarantee an optimal packing, but it keeps the packing efficient while targeting the objective of minimizing the number of canvases.

To describe the algorithm (Listing 1), we use the following notation. P is the list of polyominoes resulting from the pruning process in §5.2; C is the set of canvases currently available for packing; and h and w are the canvas height and width in tiles.

pack (Line 5) first sorts the polyominoes by size in descending order (Line 7). Then, for each polyomino, it tries to place the polyomino into the first canvas that can fit the polyomino (Line 9). `tryPlace` (Line 10) tries to place the polyomino into the canvas. It returns the first position where the polyomino can be placed, or `None` if the polyomino cannot be placed in the canvas. If the polyomino cannot be placed into any canvas, a new canvas is created and the polyomino is placed at $(0, 0)$ in the new canvas (Line 11). The overall runtime is $O(n \cdot (\log n + m \cdot h \cdot w \cdot |p|))$, where n is the number of polyominoes, m the number of canvases, and $|p|$ the largest polyomino size. After the placements are determined, Tetris renders the pixel content of the polyominoes into the canvases.

```
1  def tryPlace(p: Polyomino, c: Canvas):
2      for i in range(c.height - p.height + 1):
3          for j in range(c.width - p.width + 1):
4              if canPlaceAt(p, c, (i, j)): return i, j
5  def pack(P: list[Polyomino], h: int, w: int):
6      C: set[Canvas] = {}
7      for p in sorted(P, key=len, reverse=True):
8          position = None
9          for c in C:
10             if position := tryPlace(p, c): break
11         if position is None: C |= {c := newCanvas(h, w)}
12         markPlacement(p, c, position or (0, 0))
13         yield p, c, position or (0, 0)
```

Listing 1: FFD Polyomino Packing Algorithm

5.4 Detect, Unpack, and Track

Recall from §2.1 that N is the number of input frames and N' is the number of object detector calls; the reference pipeline issues $N' = N$ calls, one per frame. Tetris's preceding stages, relevance classification, pruning, and packing, aggregate the surviving polyominoes across frames into a set of canvases C. Because Tetris invokes the detector once per canvas, $N' = |C| \leq N$. The 3 stages therefore reduce the number of detection calls from $N' = N$ in the reference pipeline to $N' = |C|$ in Tetris, which can be substantially smaller. We now describe the remaining steps: running the detector on each packed canvas, unpacking the resulting bounding boxes to original frame coordinates, and associating detections into tracks.

5.4.1 Detection in Canvases. Tetris passes each packed canvas to the user-provided object detector. Because each canvas has the same resolution as the original video frame, objects appear at the same scale as in the reference pipeline, as the detector expects.

A single detector call on a canvas is also sufficient to recover the detections for every polyomino packed into it. The total number of detector calls is therefore reduced from N in the reference pipeline to $|C|$, which is the quantity minimized by §5.1, §5.2, and §5.3.

5.4.2 Unpacking Object Bounding Boxes. The detector outputs bounding boxes in canvas coordinates, which Tetris maps back to the original frame coordinates. For each box, Tetris identifies the polyomino containing its center within the canvas, and subtracts the polyomino's placement offset to recover its original position.

5.4.3 Tracking. The unpacked detections are in the original per-frame coordinate system, so Tetris feeds them to the tracker unchanged, producing tracks in the original video coordinate system.

6 Learned Artifacts and Pareto Frontier

Before extracting tracks on the full video dataset via the Tetris execution engine (§5), Tetris learns dataset-specific artifacts and generates a Pareto frontier of system configurations. This is done once per dataset and using small training and validation samples. Tetris chooses the training and validation samples at random; their sizes are user-settable, and our evaluation uses at most 60 one-minute videos per sample (§7). They produce the learned artifacts and a throughput-accuracy frontier, not the final tracks.

First, Tetris trains a relevance classifier specialized to the user-provided detector, which the execution engine uses to identify relevant tiles (§6.1). Second, for a given tracker, it measures empirical per-tile mistrack rates to derive $\tilde{G}^{\bar{M}}$ and passes the resulting matrix $\bar{G}^{\bar{M}}$ to the ILP for tile-granularity pruning (§6.2). Third, it generates a Pareto-optimal throughput-accuracy frontier by sweeping system configurations (§6.3), from which the user selects an operating point. After these steps, the Tetris execution engine (§5) extracts tracks on the remaining data using the selected configuration.

The second phase is the most technically challenging because we must choose tile sampling gaps without exhaustively searching all assignments. A tile sampling gap specifies how many frames may elapse before that tile must be processed again. Because different parts of a stationary scene have different motion patterns, the best gap may vary across the $H \times W$ tile grid. Exhaustively choosing one gap from Γ for every tile would require evaluating $|\Gamma|^{H \times W}$ assignments, which is computationally infeasible. Tetris therefore learns these gap choices through a scalar mistrack-rate tolerance $\bar{M} \in [0, 1]$. For each tile (i, j) and each candidate gap $\gamma \in \Gamma$, Tetris measures the empirical mistrack rate on the training sample. Given a tolerance $\bar{M}$, Tetris chooses the largest gap whose measured mistrack rate is at most $\bar{M}$ for each tile. The resulting per-tile choices form the maximum-gap matrix $\tilde{G}^{\bar{M}}$. Sweeping $\bar{M}$ produces a sequence of candidate matrices, turning the exponential gap-assignment problem into a one-dimensional sweep. Each candidate matrix is still evaluated end-to-end by running the Tetris execution engine on a held-out validation sample.

6.1 Training the Relevance Classifier

Tetris trains the relevance classifier to imitate the user-supplied object detector at the tile level, adapting the model-specialization paradigm [15] from frame-level to tile-level granularity. It runs the detector on the training video samples and labels each tile as relevant if any detector bounding box overlaps its region. Tetris trains a lightweight relevance classifier by minimizing binary cross-entropy between the predicted relevance score $Score_{f,i,j}$ (Eq. 1) and the detector-derived label. This learned classifier is the first learned artifact, used by the execution engine in §5.1 (shown in Fig. 3 A.1).

6.2 Learning Maximum Tile Sampling Gaps

The second learned artifact is $\tilde{G}^{\bar{M}}$, a matrix of maximum tile sampling gaps that lets the execution engine prune polyominoes at tile granularity. As discussed at the beginning of §6, learning $\tilde{G}^{\bar{M}}$ turns an exponential joint search over per-tile gaps into a one-dimensional sweep over the mistrack-rate tolerance $\bar{M}$. For each tile (i, j) and candidate sampling gap $\gamma \in \Gamma$, Tetris measures an empirical mistrack rate from the training sample. Given a chosen

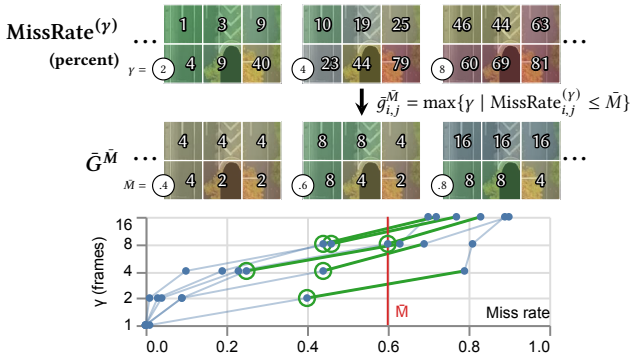


Figure 7: Mapping from measured mistrack rates to maximum tile sampling gaps on a 2×3 tile crop from B3D4. In the line chart, each line represents one tile position, connecting its measured miss rates at $\gamma \in \{1, 2, 4, 8, 16\}$ frames. For each tile, we select the largest candidate gap whose miss rate is at most $\bar{M}$. At $\bar{M} = 0.6$ tolerance boundary, each tile's point on or immediately left of it indicates its selected gap (circled).

tolerance $\bar{M}$, Tetris derives $\bar{G}^{\bar{M}}$ by selecting gaps from the measured rates under this tolerance, which the ILP (§5.2) adheres to at runtime, as illustrated in Fig. 3 A.2.

Instead of enumerating the exponential space of $|\Gamma|^{H\times W}$ gap assignments, we parameterize the selection through a single scalar mistrack-rate tolerance $\bar{M}$. At each tile, we take the largest sampling gap whose empirical mistrack rate does not exceed $\bar{M}$. In general, tiles with higher mistrack rates are assigned smaller maximum gaps, while tiles with lower mistrack rates receive larger maximum gaps. Varying $\bar{M}$ deterministically yields different learned gaps. Specifically, the measured rates define a deterministic mapping $f : [0, 1] \to \Gamma^{H\times W}$ where $f(\bar{M}) = \bar{G}^{\bar{M}}$. This collapses the combinatorial per-tile search into a one-dimensional search over $\bar{M}$, performed when generating the Pareto frontier (§6.3). In this sense, sweeping $\bar{M}$ approximates an exhaustive sweep over $\bar{G} \in \Gamma^{H\times W}$.

Given $\bar{M}$, Tetris derives $\bar{G}^{\bar{M}}$ by applying the gap-selection rule to the measured rates, as illustrated in Fig. 7. The resulting matrix has size $H \times W$ and the ILP (§5.2) strictly adheres to these maximum tile sampling gaps, defined as

$$\bar{g}^{\bar{M}}_{i,j} = \max\left\{\gamma \in \Gamma \mid \text{MissRate}^{(\gamma)}_{i,j} \le \bar{M}\right\},$$

with $\bar{g}^{\bar{M}}_{i,j}$ defaulting to 1 if no gap in Γ satisfies the criterion.

Tetris measures empirical mistrack rates as follows. First, we run the reference pipeline (§ 2.1) on the training video samples to obtain reference tracking results. It then re-runs the tracker at each candidate sampling gap in the set $\Gamma = \{1, 2, 4, 8, 16\}$, where gap γ means the tracker processes every γ-th frame. For each tile position (i, j) and sampling gap γ, Tetris counts the number of tracking associations that differ from the native-rate reference and computes a per-tile empirical mistrack rate with:

$$\text{MissRate}^{(\gamma)}_{i,j} = (\text{MissedA}^{(\gamma)}_{i,j} + 1)/(\text{TotalA}^{(\gamma)}_{i,j} + 2).$$

When sampling every γ frames, $\text{MissedA}^{(\gamma)}_{i,j}$ is the number of incorrect associations at tile (i, j), and $\text{TotalA}^{(\gamma)}_{i,j}$ counts evaluated associations at tile (i, j). Laplace smoothing avoids division by zero.

Computing $\bar{g}^{\bar{M}}_{i,j}$ as $\max\{\gamma \in \Gamma \mid \text{MissRate}^{(\gamma)}_{i,j} \le \bar{M}\}$ tile-by-tile is a heuristic approximation of the combinatorial per-tile search, as it assumes per-tile mistrack rates are independent of other tiles' gaps. We empirically show in §7.4.5 that $\bar{G}^{\bar{M}}$ lies on or near the Pareto frontier of an exhaustive sweep of all per-tile gap assignments.

6.3 Generating the Pareto Frontier

The Pareto frontier is computed using the validation sample and produces a throughput-accuracy curve, as shown in Fig. 3 B. This curve is a user-visible tuning aid rather than the final tracking output. Each point on the curve corresponds to a system parameter configuration, recording the runtime throughput and tracking accuracy achieved under that configuration. The user selects an operating point from this curve based on a throughput constraint or an accuracy constraint, and Tetris materializes the tracks on the rest of the input videos.

Tetris first executes the reference pipeline (§2.1) on the validation video samples to obtain reference tracking results. Because the input video lacks ground-truth tracks, these results serve as the tracking references against which Tetris measures the accuracy of all optimized configurations.

Additional Parameters. In addition to $\bar{M}$, T_r, and P, Tetris sweeps the whole-frame sampling rate s and the tracker choice A. The sampling rate s, described in §5, retains every s-th frame before the three Tetris operators run. Such whole-frame sampling is a common practice in video analytics systems [4, 38]. The tracker choice A determines which tracker the execution engine invokes (§5.4.3). Tetris's data model and execution engine are tracker-agnostic. The sweep considers the user-provided tracker and SORT [5], a lightweight Kalman-filter [14] tracker that Tetris uses as a built-in default. Because SORT operates only on bounding-box geometry, Tetris includes it without video-specific retraining. For each tracker option, Tetris repeats the measurement procedure of §6.2. Given a tolerance $\bar{M}$, Tetris derives the corresponding $\bar{G}^{\bar{M}}$ for the current tracker option and passes them to the ILP; the configuration sweep treats A as a discrete parameter, and each $(\bar{M}, A)$ pair yields a distinct operating point. Additional trackers can be added at the cost of additional measurement and sweep time.

Configuration Sweep. Tetris then extracts tracks via its execution engine (§5) over a sweep of parameter configurations (shown in Table 1), recording each configuration's runtime throughput and its tracking accuracy (HOTA [22]) against the reference tracking results. This step uses the measured empirical rates from §6.2 to instantiate $\bar{G}^{\bar{M}}$ for each tolerance value $\bar{M}$. Rather than sweeping maximum tile sampling gaps independently, which would require enumerating $\Gamma^{H\times W}$ possible maximum-gap matrices, the configuration sweep varies only the mistrack-rate tolerance $\bar{M}$. The mapping learned in §6.2 converts each tolerance into a full matrix $\bar{G}^{\bar{M}}$, so sweeping $\bar{M}$ acts as a tractable approximation to sweeping the exponential space of maximum-gap matrices. We employ a straightforward configuration sweep over the remaining parameter space, and Tetris's sweep covers the five parameters in Table 1.

Pareto Frontier. From all swept configurations, Tetris extracts the Pareto-optimal subset. These are configurations for which no other configuration achieves both higher throughput and higher accuracy. These Pareto-optimal points form the throughput-accuracy frontier. An example is shown by the red curve in Fig. 3 B. Users can specify a throughput constraint and Tetris selects the most accurate configuration meeting it, or an accuracy constraint and Tetris selects the highest-throughput configuration meeting it, or pick a configuration directly from the frontier.

Once the user selects an operating point, the chosen configuration and the learned artifacts are passed to the Tetris execution engine (§ 5), which extracts tracks on the remaining video data. Tetris retains every s-th frame from the input video and discards the rest. On each retained frame, the specialized relevance classifier (§6.1) scores tiles against threshold T_r. The polyomino-pruning ILP (§5.2) enforces the maximum tile sampling gaps $\bar{G}^{\bar{M}}$ learned in §6.2, derived from the chosen tolerance $\bar{M}$. Surviving polyominoes are padded with the pattern P and packed into canvases (§5.3). Finally, detection runs on the canvases and the selected tracker A produces the tracks for the full dataset, as shown in Fig. 3 C.

7 Evaluation

We evaluate Tetris's efficiency in tracking objects while maintaining tracking fidelity, comparing against prior systems on seven datasets.

Accuracy Metrics. Prior systems evaluate accuracy using predefined queries [4, 15, 37], which can restrict evaluation to anticipated situations and fail to capture more complex object behaviors. We instead adopt HOTA (§ 2.2) as a query-agnostic measure of track reconstruction quality. HOTA balances detection accuracy with trajectory association accuracy, so missing detections, fragmented tracks, and merged identities all reduce the score.

Datasets. We evaluate Tetris on seven video datasets (CalDoT1/2, B3D1/2/3/4, and Amsterdam) that vary in scene density, camera viewpoint, and object motion patterns, allowing us to test performance across a diverse set of real-world conditions. CalDoT1/2 (720 × 480 pixels) were introduced by OTIF [4], while Amsterdam (1280 × 720 pixels) was introduced by NoScope [15]. We add the B3D (1080 × 720 pixels) datasets (B3D1/2/3/4) to expand the evaluation beyond existing benchmarks. All datasets consist of approximately one-minute clips captured at 15 FPS; CalDoT1/2, and Amsterdam each include 180 clips, while each B3D dataset includes 18 clips.

CalDoT1/2 [4] consist of highway traffic footage from California Department of Transportation cameras; Amsterdam [15] is captured from an urban street-side camera. These datasets exhibit structured motion in relatively constrained settings. The B3D datasets are drawn from the Berkeley DeepDrive Drone Dataset [35], consisting of aerial footage of unsignalized intersections and roundabouts with less predictable motion and higher interaction density. We mask non-road and parking areas in the B3D videos. By including B3D, we extend beyond the structured highway and low-density roadway settings of CalDoT and Amsterdam.

For CalDoT1/2 and Amsterdam, we partition the data into training, validation, and testing splits, each consisting of 60 one-minute clips. For the B3D datasets, because of the amount of footage available, we use training, validation, and testing splits of 8, 5, and 5 approximately one-minute clips, respectively.

Ground Truth. We obtain ground truth detection and tracking annotations for the test splits used in our end-to-end evaluation (§7.1). For CalDoT1 and CalDoT2, we use the dense per-frame object detection labels provided by the OTIF dataset. For Amsterdam, we start from the same OTIF dense labels, filter to vehicle classes only, and apply minor manual corrections to remove erroneous annotations. For B3D1/2/3/4, we generate dense detection labels using Faster R-CNN [30] trained with training labels from the B3D paper. Ground truth tracking annotations for all datasets are generated using OC-SORT [6]. The B3D automated detection and tracking labels are verified and corrected by the authors.

In our experiments, the user-provided tracker is BYTETrack [39], which also serves as the reference while generating the Pareto frontier. OC-SORT is used only for ground truth generation, ensuring that evaluation results are not biased by shared trackers. Tetris does not have access to the ground-truth labels while learning the artifacts or generating the Pareto frontier.

Prior Systems. We compare Tetris against two prior approaches: OTIF and LEAP. OTIF utilizes a lightweight proxy model to identify irrelevant regions within frames for more efficient batching, along with uniform frame sampling and a specialized tracker to further reduce calls to the object detector. LEAP uses predictive sampling to prune frames that are unlikely to contain relevant information. For both methods, we use the authors' available implementations.

For OTIF, we use the training split for its recurrent rate tracker, since the original "tracker" split is unavailable across all datasets.

For a fair comparison across systems, we standardize the object detector used in all experiments. Specifically, we use YOLOv5 [16] for the CalDoT1, CalDoT2, and Amsterdam datasets, and RetinaNet [20] for the B3D datasets, i.e., the detector released with B3D. This ensures that any throughput and accuracy differences arise from the system rather than underlying detection models. For Tetris, the user-provided tracker is BYTETrack and the built-in default is SORT, so Pareto frontier generation searches over the two tracker options {BYTETrack, SORT}. When generating the Pareto frontier (§ 6.3), BYTETrack at full frame rate serves as the user-provided tracker in the reference pipeline (§ 2.1), and the Pareto-optimal operating points may draw from either tracker option.

Detector Training. We train the YOLOv5 models on the training splits using the same curated labels described above: the dense detection labels from the OTIF dataset for CalDoT1 and CalDoT2, and the vehicle-filtered, manually corrected OTIF labels for Amsterdam.

Experiment Setup. All evaluated systems consist of two stages: a preparation stage to learn from the training split and generate their configurations using the validation split, and a track extraction stage to track objects in the test split. We report throughput on track extraction only. For Tetris, this includes relevance classification, polyomino grouping, ILP construction and solving, packing, detection, unpacking, tracking, and interpolation. Each data point in the results corresponds to a distinct system configuration. For Tetris and OTIF, configurations are the Pareto-optimal operating points selected on the validation set (§ 6.3). LEAP, having a single fixed configuration, contributes one point. For Tetris and OTIF, we report only the Pareto-optimal points from the test split results.

Hardware. All experiments are conducted on a server equipped with dual Intel Xeon Gold 6248 CPUs, (2.50 GHz) 504 GB of RAM,

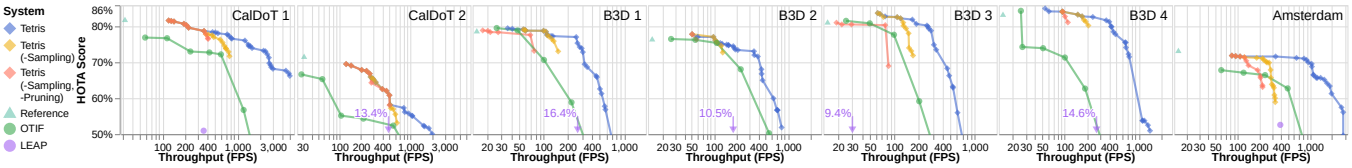


Figure 8: End-to-end tracking accuracy versus throughput for Tetris, the reference pipeline, prior systems, and ablations.

and an NVIDIA Quadro RTX 6000 GPU (24 GB VRAM). Reported runtimes measure computation time only, excluding disk I/O.

Track Interpolation. Tracking outputs from all systems are linearly interpolated to produce detection points on every frame for every track. This step ensures a fair comparison under the HOTA metric, which penalizes missing per-frame detections within tracks. Systems that employ frame skipping inherently produce tracks with detection gaps; without interpolation, these gaps would exaggerate the HOTA penalty even when the underlying motion is faithfully captured, for example, a vehicle traveling at constant speed along a straight path. By applying interpolation uniformly across all systems, we isolate tracking quality from the detection gaps that frame-sampling strategies introduce.

7.1 End-to-end Results

Fig. 8 summarizes the throughput-accuracy tradeoff across all seven datasets. Tetris consistently dominates the Pareto frontier, achieving higher HOTA scores than both OTIF and LEAP at the same throughput, and higher throughput at the same accuracy level. At matched throughput, we compare each prior work's Pareto-optimal configuration against the most accurate Tetris configuration that is at least as fast. Tetris achieves higher HOTA in 45 of these 46 comparisons (out of 54 prior works' configurations). In the remaining comparison, on B3D1, the prior configuration is ahead by 0.001 HOTA. Of the remaining 8 prior configurations that exceed Tetris's highest throughput, 7 produce zero HOTA; the last achieves a 1.3× speedup over Tetris's fastest configuration while dropping HOTA by 0.20 on the B3D2 dataset. The median HOTA gain across these comparisons is 0.09, per-dataset maximum gains range from 0.10 to 0.42, and the largest gain occurs at 1000 FPS on the B3D4 dataset.

In realistic deployment scenarios where tracking accuracy is critical, we measure the maximum speedup each system achieves over the full-frame, every-frame reference pipeline at bounded HOTA accuracy loss. Tetris achieves a speedup while staying within a 5% HOTA drop from the reference pipeline on all 7 datasets, while prior systems meet this constraint on only 4/7 datasets. To avoid overclaiming Tetris's improvement over prior systems, we restrict prior systems to Pareto-optimal configurations and compare against a prior configuration whose HOTA is no higher than Tetris's selected configuration. Under this 5% constraint, Tetris achieves 1.5× to 17.4× speedup over prior systems, and 6.5× to 67.5× speedup over the reference pipeline. At a 10% HOTA drop, Tetris achieves 1.6× to 24.7× speedup over prior systems, and 10.2× to 71.8× speedup over the reference pipeline. Under both constraints, Tetris consistently achieves higher speedup than all other systems on every dataset.

Analysis. LEAP is designed to optimize for object-existence queries, such as counting or detecting the time period during which an object is present, rather than reconstructing full per-frame object tracks. Accordingly, LEAP outputs a time interval and an assigned motion pattern for each detected object rather than a per-frame trajectory. For LEAP's HOTA evaluation, we consider two approaches: (i) trajectories reconstructed from sampled detection boxes, and (ii) trajectories reconstructed from LEAP's own assigned motion patterns and presence intervals. LEAP learns its motion patterns from the training clips during preparation. In (ii), each object keeps its LEAP identity, and its assigned pattern is placed over its predicted presence interval, linearly resampled when shorter than the interval and truncated when longer. Neither approach uses ground-truth or reference-pipeline tracks. We use the highest-HOTA approach for each dataset in comparisons with Tetris and OTIF, giving LEAP its best result. LEAP achieves its highest HOTA on CalDoT1 and Amsterdam, where small position changes between consecutive sampled observations aid reconstruction from sampled detections.

Among CalDoT2 vehicles that enter and exit within a clip and remain visible for at least one second, 40.9% of those detected by LEAP are observed only once. Among those observed at least twice, 41.6% receive different identities at their first two observations.

B3D captures unsignalized intersections and traffic circles, where varied routes and vehicle visibility durations can limit motion-pattern prediction and predictive sampling. For vehicles that enter and exit within a clip and remain visible for at least one second, the pooled 10th–90th percentile visibility durations span 8.4–25.9 s.

On the other hand, OTIF achieves reasonable accuracy, but its speedup derives primarily from uniform frame skipping, with its segmentation proxy model contributing comparatively little to throughput gains, as reported by Bastani et al. [4]. This reliance on frame skipping produces coarse-grained tracks that degrade accuracy at higher throughput operating points, consistent with the observation that uniform temporal sampling cannot accommodate spatially varying sampling requirements (§2.4, Obs. 2). The limited benefit of OTIF's proxy is visible directly in the B3D4 Pareto front. Its most accurate point disables the segmentation proxy entirely, while the next point enables it and exhibits a sharp accuracy drop with little speedup gain. This points to a limitation that motivates Tetris's design: OTIF's rectangular crops are a poor fit for the irregular shape of real activity regions (§2.4, Obs. 3), forcing the proxy to either over-include background (yielding little speedup) or discard tiles with detections (yielding the observed accuracy drop).

7.2 Preparation Cost and Amortization

Fig. 9 Left compares the preparation each system needs before extraction, split into training and validation. For Tetris, training creates reference labels and learned artifacts (§6.1, §6.2); validation

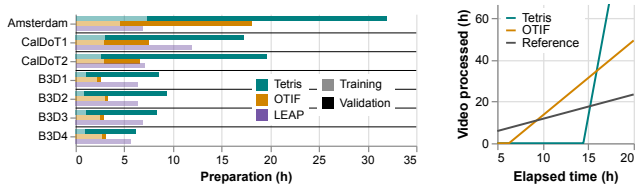


Figure 9: Left Preparation time split into training and validation. Right Video processed against elapsed time including preparation, for Tetris's 5% HOTA-loss configuration, the paired OTIF configuration, and the reference pipeline.

Table 2: Query scores (%) and extraction FPS, each listed as Tetris / OTIF / LEAP. Best displayed values are bold.

Dataset	Selection ↑	Count ↓	Top-10 ↑	FPS ↑
CalDoT1	95.9 / **96.8** / 95.4	10.9 / **10.0** / 33.0	**100** / 90.0 / 90.0	**458** / 422 / 354
CalDoT2	94.5 / **97.5** / 91.5	22.1 / **15.4** / 42.0	**90.0** / **90.0** / **90.0**	**493** / 478 / 487
B3D1	**100** / 99.9 / **100**	**20.1** / 38.7 / 43.9	80.0 / 70.0 / **90.0**	**281** / 223 / 265
B3D2	**100** / 99.5 / **100**	**25.8** / 28.9 / 27.3	50.0 / **70.0** / 50.0	**346** / 230 / 182
B3D3	**100** / **100** / **100**	8.6 / **5.9** / 96.1	**80.0** / **80.0** / **80.0**	**31** / 25 / 30
B3D4	**100** / 99.0 / **100**	**15.7** / 39.5 / 124	**100** / 80.0 / 60.0	**363** / 188 / 268
Amsterdam	**98.3** / 97.9 / 98.2	**19.9** / 23.3 / 59.3	90.0 / **100** / 90.0	**581** / 492 / 389

generates the Pareto frontier by running and scoring all configurations. OTIF trains its proxy model and tracker, then tunes configurations on validation videos. LEAP trains its tracker model and learns motion patterns, without a validation search. Validation accounts for 77–91% of Tetris's preparation, while classifier training takes only 4–12% and the detector pass over the training split takes 1–3%. In total, Tetris needs 6.1–31.9 hours, compared with 2.5–18.1 hours for OTIF and 5.7–12.0 hours for LEAP.

Fig. 9 Right charges preparation before extraction, averaged over datasets. A system with preparation P_h hours and throughput F_s FPS has processed $F_s \max(0, t - P_h)/15$ hours of 15-FPS video after t hours. At the 5% HOTA-loss configurations, Tetris overtakes the reference pipeline after 6.4–32.4 hours. At the matching 5% loss configuration (§7.1), Tetris overtakes OTIF after 6.3–32.8 hours.

LEAP only has 1 configuration, and never reaches the 5% bound on any dataset; we separately compare it against Tetris's fastest configuration that is more accurate than LEAP. Tetris overtakes LEAP after 6.3–35.9 hours.

Tetris spends preparation to gain extraction throughput. It recovers this cost within 35.9 hours of elapsed time on every dataset, after which it processes video 6.5–67.5× faster than the reference pipeline and 1.5–17.4× faster than the paired OTIF configuration.

7.3 Downstream Query Accuracy

To assess Tetris on LEAP's intended workload, we compare the three systems on LEAP's frame-selection, per-frame counting, and top-10 queries [37]. We report F1, MAPE [25], and top-10 precision, respectively, treating all vehicles as one class. LEAP uses its native presence intervals; Tetris and OTIF use their materialized tracks.

We choose OTIF's test configuration closest to LEAP in FPS and Tetris's closest test configuration that is faster than both baselines (Table 2). At these faster operating points, Tetris is best in 61.9% of dataset–query cases, comparable (within 1 percentage point) in

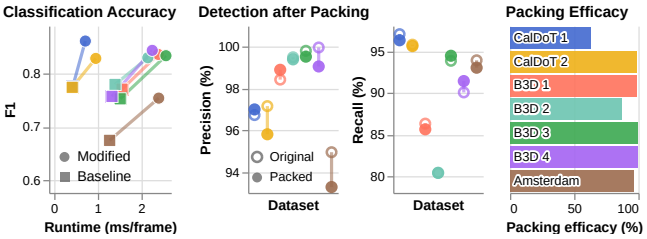


Figure 10: Left Relevance-classifier comparison. Modified: §5.1. Baseline: RGB-only ShuffleNet [24]. Middle Detector precision and recall on original frames and packed canvases per dataset. Right Packing efficacy, also as color legend.

9.5%, behind the best baseline by at most 5 points in 9.5%, 20 points in only 1 case, and by at most 10 points in the remaining 14.3%.

These results also support our goal of efficiently materializing accurate tracks for reuse across track-related queries.

7.4 Ablation Studies

We now systematically break down the contribution of the operators in Tetris's tracking pipeline. We first evaluate the relevance classifier against an RGB-only baseline, then measure how packing affects detector accuracy and how densely the packer fills the canvases. We examine the throughput improvements achieved by sequentially enabling our core spatial and temporal optimization operators, and then whole-frame sampling. We also validate the one-dimensional tolerance sweep against the full combinatorial per-tile gap search. Finally, we break down the extraction runtime by operator and evaluate how the ILP scales.

7.4.1 Relevance Classifier Accuracy. We compare the deployed relevance classifier against the original RGB-only ShuffleNet baseline. Across 7 datasets at T_r=0.50, adding the frame-difference image and position encoding improves F1 over the baseline on all 7 datasets, as shown in the Left panel of Fig. 10. The augmented classifier yields an average F1 gain of 7.15%. The largest gain appears on B3D 4, where F1 rises from 0.7571 to 0.8431 (gain 8.60%). This gain comes with 76.4% higher classifier inference time on average.

7.4.2 Detection Accuracy after Packing. Packing places polyominoes from different frames and image locations side by side, changing the detector's input distribution. We quantify how packing affects detection accuracy on the test split. To isolate its effect, we select relevant tiles using ground-truth boxes and disable ILP pruning. We sample test frames at s=16 and use the padding from each dataset's 5% HOTA-loss configuration (§7.1), with P=.5 on Amsterdam and CalDoT and P=0 on B3D. We apply the reference detector to the original sampled frames and their packed canvases, then map canvas detections back to source-frame coordinates. We score both outputs against ground truth, including only annotations and detections whose centers lie in retained tiles. Precision and recall use one-to-one matching at IoU ≥ .5, with detections processed in descending confidence order.

With these padding settings, packing changes precision and recall by at most 1.7 percentage points (Middle panels of Fig. 10).

On Amsterdam and CalDoT, packing without padding lowers recall by 3.2–9.9 points relative to detection on the original frames.

7.4.3 Packing Efficacy. We quantify how tightly Tetris's packer fills each generated canvas by measuring the fraction of total canvas tile-cells that the packer actually uses, packing efficacy $= \frac{\text{occupied tiles}}{\text{occupied tiles+empty tiles}}$, where the denominator is the number of canvases × canvas height × canvas width, measured in tile units. We use classifier-selected tiles on the test split with s=1, T_r=0.5, $\bar{M}$=none, and P=0. The packer utilizes 91.86% of the available canvas area on average across 7 datasets, peaking at 99.60% on B3D 3. The Right panel of Fig. 10 shows the per-dataset packing efficacy.

The two least densely packed datasets are B3D 2 and CalDoT 1, which fall noticeably below the average (down to 63.02%). We traced both to dataset-specific properties of the relevant polyominoes. B3D 2 captures a busy unsignalized intersection in which vehicles routinely cluster shoulder-to-shoulder. Adjacent tiles with vehicles merge into a single connected polyomino, so the per-canvas polyomino population is shifted toward fewer, larger polyominoes than on the less-congested B3D scenes. Specifically, aggregated across all test videos, B3D 2 produces 31.8 polyominoes per canvas averaging 5.92 tiles each, versus 61.3 polyominoes per canvas averaging 3.51 tiles each on B3D 3. CalDoT 1 exhibits a similar effect. Aggregated across all test videos, CalDoT 1 produces 15.2 polyominoes per canvas averaging 3.98 tiles each, versus 39.3 polyominoes per canvas averaging 2.40 tiles each on CalDoT 2.

7.4.4 Optimization Operators. We measure the impact of each optimization technique by incrementally enabling them and comparing their throughput over the reference baseline under a strict 5% maximum HOTA loss. Tetris first introduces the relevance classification and polyomino packing operators. As shown by the Pareto front results, this combined spatial pruning approach brings the throughput speedup over the reference execution up to 12.0× (average 6.9×) while safely remaining within the tracking accuracy constraint. Next, Tetris adds the fine-grained polyomino pruning operator, which significantly accelerates tracking in simple, low-activity video regions. The inclusion of polyomino pruning further increases the maximum speedup to 17.6× (average 11.3×) over the baseline execution. Finally, we evaluate enabling the whole-frame sampling knob $s > 1$ (§6.3). Together, the complete execution engine achieves a maximum speedup of 67.5× (average 26.7×) while still strictly adhering to the 5% tracking accuracy loss constraint.

7.4.5 Maximum tile sampling gaps. We evaluate the sweep over the mistrack-rate tolerance $\bar{M}$ from §6.2. Each tolerance yields a different $\bar{G}^{\bar{M}}$. We show that the sweep approximates the Pareto frontier of pruning ratio versus HOTA against obtained by exhaustively enumerating all possible $\bar{G}$. To keep the exhaustive baseline feasible, we reduce the tile grid to 3 × 3 and the candidate gap set to $\{1, 2, 4\}$, yielding $3^9 \approx 20{,}000$ per-tile gap assignments. We do not report throughput directly on this sweep, because re-running full tracking execution on every assignment is prohibitively expensive; instead, we use the pruning ratio (fraction of tiles skipped) as a throughput proxy. We use BYTETrack as the tracker.

To isolate the effect of the maximum tile sampling gaps from classifier and detector variance, the experimental pipeline constructs polyominoes from the reference-pipeline bounding boxes rather than classifier output, retains only boxes inside a polyomino retained on the same frame, and feeds them to BYTETrack. We report HOTA against the ground-truth tracks alongside the pruning ratio.

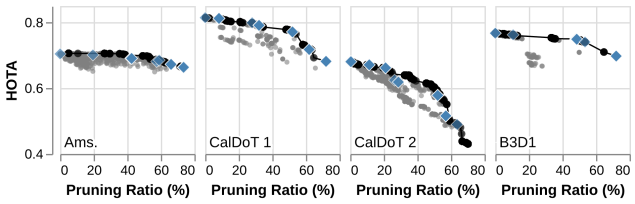


Figure 11: HOTA vs. pruning ratio per dataset. Gray: exhaustive per-tile sampling assignments over a 3 × 3 grid with gaps $\{1, 2, 4\}$. Black: Pareto frontier of the exhaustive sweep. Blue diamonds: our heuristic $\bar{G}^{\bar{M}}$ swept over $\bar{M}$ with BYTETrack.

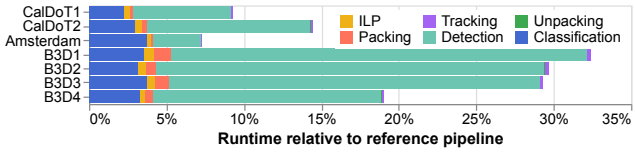


Figure 12: Runtime breakdown. Each segment is the operator's counted runtime as a percentage of the reference pipeline's total runtime on the same clips.

Fig. 11 shows the resulting trade-off per dataset. The heuristic points (blue diamonds), obtained by sweeping $\bar{M}$, lie on or near the Pareto frontier of the exhaustive per-tile sweep across datasets, indicating that the one-dimensional tolerance sweep loses little relative to the full combinatorial search. Quantitatively, at each heuristic pruning level we compare against the exhaustive Pareto point with the highest pruning ratio not exceeding that level; HOTA loss versus that anchor is at most 8.27% (mean 0.78%).

7.4.6 Extraction Operator Costs Breakdown. Fig. 12 reports runtime breakdown for the configuration (s=1, T_r=0.5, $\bar{M}$=0.2, P=1, SORT) across datasets. Classification takes 10.6–51.4% of total extraction, as it reduces downstream detection work. Running classification on every frame, however, costs only 2.3–3.8% of all-frame detector compute. Both the classifier and the detector process every pixel at native resolution, so this overhead should stay similar for higher-resolution video. Optimistically eliminating all other work gives a classifier-only speedup ceiling of 26.4–44.3× at s=1 and 423–708× at s=16. Pruning, including ILP construction and solving, takes 1.6–4.4% of extraction, or 0.28–0.66% of reference-pipeline time. Before presolve, the largest ILP in each dataset has 1,603–9,165 variables and 6,815–51,063 constraints. Packing takes 1.8–3.3% of extraction, or 0.14–1.08% of reference-pipeline time, mostly canvas rendering (64.5–83.9% of packing time).

7.4.7 ILP Scalability. Since each ILP covers a one-minute window of video (§5.2), its size depends on the relevant tiles in that window rather than on video length. Tetris limits each ILP to a 0.1-s solver budget. Under the same configuration as § 7.4.6 and across the 200 clips, 1,356,321 relevant tiles enter the ILP, and pruning keeps

562,071 of them. 97 of the 200 ILPs finish within the budget. To estimate the pruning this budget leaves unrealized, we rerun the same ILPs with a 60-s budget. In this rerun, 120 return within 1 s and 137 within 10 s, while 48 reach the cap. Even treating the capped results as final, the 60-s budget keeps only 0.4% fewer tiles (2,232 tiles). The ILP's solve time therefore stays bounded per minute of video, with little pruning lost.

8 Conclusion

We presented Tetris, a track-extraction system that decomposes stationary-camera video into a tile-based *polyomino* data model, enabling fine-grained spatiotemporal pruning that reduces detector calls with minimal fidelity loss. Our evaluation across seven video datasets shows that Tetris consistently stays within a 5% HOTA loss on all datasets, while prior systems exceed this bound on 3 of them. Tetris achieves up to 17.4× higher throughput than prior systems and up to 67.5× higher than the reference pipeline.

References

[1] Brenda S Baker. 1985. A new proof for the first-fit decreasing bin-packing algorithm. *Journal of Algorithms* 6, 1 (1985), 49–70. doi:10.1016/0196-6774(85)90018-5

[2] Jaeho Bang, Gaurav Tarlok Kakkar, Pramod Chunduri, Subrata Mitra, and Joy Arulraj. 2023. Seiden: Revisiting Query Processing in Video Database Systems. *Proc. VLDB Endow.* 16, 9 (May 2023), 2289–2301. doi:10.14778/3598581.3598599

[3] Favyen Bastani, Songtao He, Arjun Balasingam, Karthik Gopalakrishnan, Mohammad Alizadeh, Hari Balakrishnan, Michael Cafarella, Tim Kraska, and Sam Madden. 2020. MIRIS: Fast Object Track Queries in Video. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (Portland, OR, USA) *(SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 1907–1921. doi:10.1145/3318464.3389692

[4] Favyen Bastani and Samuel Madden. 2022. OTIF: Efficient Tracker Pre-processing over Large Video Datasets. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 2091–2104. doi:10.1145/3514221.3517835

[5] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. 2016. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*. 3464–3468. doi:10.1109/ICIP.2016.7533003

[6] Jinkun Cao, Jiangmiao Pang, Xinshuo Weng, Rawal Khirodkar, and Kris Kitani. 2023. Observation-Centric SORT: Rethinking SORT for Robust Multi-Object Tracking. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 9686–9696. doi:10.1109/CVPR52729.2023.00934

[7] Tiffany Yu-Han Chen, Lenin Ravindranath, Shuo Deng, Paramvir Bahl, and Hari Balakrishnan. 2015. Glimpse: Continuous, Real-Time Object Recognition on Mobile Devices. In *Proceedings of the 13th ACM Conference on Embedded Networked Sensor Systems*. doi:10.1145/2809695.2809711

[8] M. R. Garey and D. S. Johnson. 1977. The Rectilinear Steiner Tree Problem is NP-Complete. *SIAM J. Appl. Math.* 32, 4 (1977), 826–834. doi:10.1137/0132071

[9] Michael R. Garey and David S. Johnson. 1990. *Computers and Intractability; A Guide to the Theory of NP-Completeness*. W. H. Freeman & Co., USA.

[10] Ila Gokarn, Hemanth Sabella, Yigong Hu, Tarek Abdelzaher, and Archan Misra. 2023. MOSAIC: Spatially-Multiplexed Edge AI Optimization over Multiple Concurrent Video Sensing Streams. In *Proceedings of the 14th ACM Multimedia Systems Conference*. doi:10.1145/3587819.3590986

[11] S.W. Golomb. 1996. *Polyominoes: Puzzles, Patterns, Problems, and Packings*. Princeton University Press. https://books.google.com/books?id=sbQj2dILLIsC

[12] Klaus Jansen, Stefan Kratsch, DáNiel Marx, and Ildikó Schlotter. 2013. Bin packing with fixed number of bins revisited. *J. Comput. Syst. Sci.* 79, 1 (Feb. 2013), 39–49. doi:10.1016/j.jcss.2012.04.004

[13] D. S. Johnson, A. Demers, J. D. Ullman, M. R. Garey, and R. L. Graham. 1974. Worst-Case Performance Bounds for Simple One-Dimensional Packing Algorithms. *SIAM J. Comput.* 3, 4 (Dec. 1974), 299–325. doi:10.1137/0203025

[14] R. E. Kalman. 1960. A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering* 82, 1 (03 1960), 35–45. doi:10.1115/1.3662552

[15] Daniel Kang, John Emmons, Firas Abuzaid, Peter Bailis, and Matei Zaharia. 2017. NoScope: optimizing neural network queries over video at scale. *Proc. VLDB Endow.* 10, 11 (Aug. 2017), 1586–1597. doi:10.14778/3137628.3137664

[16] Rahima Khanam and Muhammad Hussain. 2024. What is YOLOv5: A deep look into the internal features of the popular object detector. arXiv:2407.20892 [cs.CV] https://arxiv.org/abs/2407.20892

[17] Donald E. Knuth. 2000. Dancing links. arXiv:cs/0011047 [cs.DS] https://arxiv.org/abs/cs/0011047

[18] Ferdinand Kossmann, Ziniu Wu, Eugenie Lai, Nesime Tatbul, Lei Cao, Tim Kraska, and Sam Madden. 2023. Extract-Transform-Load for Video Streams. *Proc. VLDB Endow.* 16, 9 (2023), 2302–2315. doi:10.14778/3598581.3598600

[19] Yuanqi Li, Arthi Padmanabhan, Pengzhan Zhao, Yufei Wang, Guoqing Harry Xu, and Ravi Netravali. 2020. Reducto: On-Camera Filtering for Resource-Efficient Real-Time Video Analytics. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication (SIGCOMM '20)*. Association for Computing Machinery, New York, NY, USA, 359–376. doi:10.1145/3387514.3405874

[20] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal Loss for Dense Object Detection. In *2017 IEEE International Conference on Computer Vision (ICCV)*. 2999–3007. doi:10.1109/ICCV.2017.324

[21] Andrea Lodi, Silvano Martello, and Daniele Vigo. 2002. Recent advances on two-dimensional bin packing problems. *Discrete Applied Mathematics* 123, 1 (2002), 379–396. doi:10.1016/S0166-218X(01)00347-X

[22] Jonathon Luiten, Aljoša Ošep, Patrick Dendorfer, Philip Torr, Andreas Geiger, Laura Leal-Taixé, and Bastian Leibe. 2021. HOTA: A Higher Order Metric for Evaluating Multi-object Tracking. *Int. J. Comput. Vision* 129, 2 (Feb. 2021), 548–578. doi:10.1007/s11263-020-01375-2

[23] Wenhan Luo, Junliang Xing, Anton Milan, Xiaoqin Zhang, Wei Liu, and Tae-Kyun Kim. 2021. Multiple object tracking: A literature review. *Artificial Intelligence* 293 (2021), 103448. doi:10.1016/j.artint.2020.103448

[24] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. 2018. ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design. arXiv:1807.11164 [cs.CV] https://arxiv.org/abs/1807.11164

[25] Spyros Makridakis, S. Wheelwright, and Rob Hyndman. 1984. *Forecasting: Methods and Applications*. Vol. 35. doi:10.2307/2581936

[26] Kara Manke. 2022. Massive traffic experiment pits machine learning against 'phantom' jams. Berkeley News. https://news.berkeley.edu/2022/11/22/massive-traffic-experiment-pits-machine-learning-against-phantom-jams/

[27] Thanh-Tung Nguyen, Si Young Jang, Boyan Kostadinov, and Dongman Lee. 2023. PreActo: Efficient Cross-Camera Object Tracking System in Video Analytics Edge Computing. In *2023 IEEE International Conference on Pervasive Computing and Communications (PerCom)*. 101–110. doi:10.1109/PERCOM56429.2023.10099298

[28] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. 2016. You Only Look Once: Unified, Real-Time Object Detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 779–788. doi:10.1109/CVPR.2016.91

[29] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. 2015. Faster R-CNN: towards real-time object detection with region proposal networks. In *Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 1* (Montreal, Canada) *(NIPS'15)*. MIT Press, Cambridge, MA, USA, 91–99.

[30] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. 2016. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. arXiv:1506.01497 [cs.CV] https://arxiv.org/abs/1506.01497

[31] Leslie G. Valiant. 1981. Universality Considerations in VLSI Circuits. *IEEE Trans. Comput.* C-30, 2 (1981), 135–140. doi:10.1109/TC.1981.6312176

[32] Weijun Wang, Liang Mi, Shaowei Cen, Haipeng Dai, Yuanchun Li, Xiaoming Fu, and Yunxin Liu. 2025. Region-based Content Enhancement for Efficient Video Analytics at the Edge. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, 613–633. https://www.usenix.org/conference/nsdi25/presentation/wang-weijun

[33] Nicolai Wojke and Alex Bewley. 2018. Deep Cosine Metric Learning for Person Re-identification. In *2018 IEEE Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 748–756. doi:10.1109/WACV.2018.00087

[34] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. 2017. Simple Online and Realtime Tracking with a Deep Association Metric. In *2017 IEEE International Conference on Image Processing (ICIP)*. IEEE, 3645–3649. doi:10.1109/ICIP.2017.8296962

[35] Fangyu Wu, Dequan Wang, Minjune Hwang, Chenhui Hao, Jiawei Lu, Jiamu Zhang, Christopher Chou, Trevor Darrell, and Alexandre Bayen. 2025. Decentralized Vehicle Coordination: The Berkeley DeepDrive Drone Dataset and Consensus-Based Models. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*. 2438–2444. doi:10.1109/ICRA55743.2025.11127341

[36] Yuheng Wu, Thanh-Tung Nguyen, Lucas Liebe, Quang Tau, Pablo Espinosa Campos, Jinghan Cheng, and Dongman Lee. 2025. How2Compress: Scalable and Efficient Edge Video Analytics via Adaptive Granular Video Compression. In *Proceedings of the 33rd ACM International Conference on Multimedia*. doi:10.1145/3746027.3754922

[37] Yanchao Xu, Dongxiang Zhang, Shuhao Zhang, Sai Wu, Zexu Feng, and Gang Chen. 2024. Predictive and Near-Optimal Sampling for View Materialization in Video Databases. *Proc. ACM Manag. Data* 2, 1, Article 19 (March 2024), 27 pages. doi:10.1145/3639274

[38] Baiyan Zhang, Zepeng Li, Dongxiang Zhang, Huan Li, Kian-Lee Tan, and Gang Chen. 2025. High-Throughput Ingestion for Video Warehouse: Comprehensive Configuration and Effective Exploration. *Proc. ACM Manag. Data* 3, 3, Article 169 (June 2025), 27 pages. doi:10.1145/3725407

[39] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. 2022. ByteTrack: Multi-Object Tracking by Associating Every Detection Box. (2022).

[40] Andong Zhu, Sheng Zhang, et al. 2024. Crucio: End-to-End Coordinated Spatio-Temporal Redundancy Elimination for Fast Video Analytics. In *IEEE INFOCOM 2024*. doi:10.1109/INFOCOM52122.2024.10621116

A NP-hardness Proofs

We now prove the NP-hardness of the two optimization problems described in §5: polyomino pruning (§5.2) and polyomino packing (§5.3).

A.1 Polyomino Pruning is NP-hard

Recall the pruning problem from §5.2. Given N frames of relevant polyominoes and a maximum tile sampling gap $\bar{g}^{M}_{i,j}$ for each tile location, we select a subset of the polyominoes that minimizes the total number of selected tiles, such that every window of $\bar{g}^{M}_{i,j}$ consecutive frames containing a polyomino that covers (i, j) also contains a selected polyomino covering (i, j).

We prove NP-hardness in two stages. First (§A.1.1), we define a relaxed variant of polyomino pruning in which some tiles do not enforce a sampling gap, denoted $\bar{g}^{M}_{i,j} = \text{null}$; such tiles need not be covered by any selected polyomino. We show that this relaxed variant can encode the Minimum Vertex Cover problem on planar graphs with maximum degree of three, which is NP-hard [8, 9]. Second (§A.1.2), we show that polyomino pruning as originally formulated, where every tile enforces a sampling gap, can encode the relaxed variant, so the hardness carries over.

A.1.1 Relaxed Polyomino Pruning Encodes Minimum Vertex Cover. In the pruning instance that the reduction constructs, selecting a polyomino corresponds to placing a vertex in the cover, one tile per edge carries a sampling gap that only the polyominoes of the edge's two endpoints can satisfy, and the minimum tile objective minimizes the number of selected vertices.

Step 1: The graph as a tile grid. Given such a graph $G = (V, E)$ with $n = |V|$ vertices, we lay the graph out on a tile grid whose dimensions $R \times C$ are polynomial in n, with edges routed along grid lines [31], as shown in Fig. 13. Each vertex occupies one tile, each edge becomes a path of tiles between its two endpoint vertices' tiles, and every edge path occupies at least one tile strictly between its two endpoints; we mark one such interior tile m_e on each edge path as the *midpoint* of edge e. This layout represents the whole graph spatially on a single tile grid of size $R \times C$.

Step 2: Vertices as polyominoes. We next represent each vertex v as a single connected polyomino S_v on this tile grid. S_v covers

(1) vertex v's tile,
(2) the midpoint tile m_e of every edge e incident to v,
(3) and the edge-path tiles that connect v's tile to each of these midpoints.

S_v thus consists of v's tile at its center and one arm per incident edge; we call S_v the *vertex polyomino* of v. Since each edge has exactly two endpoints, each midpoint m_e appears in exactly two vertex polyominoes, those of e's two endpoints. Together, the n vertex polyominoes represent the graph: each vertex v becomes a polyomino S_v, and each edge (u, v) corresponds to its midpoint $m_{(u,v)}$, the only tile that S_u and S_v share. The bottom row of Fig. 14 shows the vertex polyominoes of the triangle graph K_3.

Step 3: Placing the polyominoes into frames. We next spread this representation over time. We create n video frames, one frame f_v per vertex v, each its own $R \times C$ tile grid; the video thus contains n tile grids, and the single grid of Step 1 is not one of them but the shared layout from which every frame takes its coordinates.

We place each vertex polyomino S_v alone on its frame f_v, at the same tile coordinates it occupies in the layout of Step 1: v's tile lies at its layout location, as does the midpoint $m_{(u,v)}$ of every edge (u, v) incident to v. Each midpoint $m_{(u,v)}$ thus appears at the same tile location in both f_u and f_v, whose polyominoes S_u and S_v cover it.

In the original problem's data model, polyominoes on the same frame must be disjoint (§4.2), so S_u and S_v, which share the midpoint tile $m_{(u,v)}$, could not coexist on a single frame; spreading the polyominoes over time is what lets two of them cover the same tile location.

Step 4: Padding polyominoes to a common size. The vertex polyominoes generally differ in size, since arm lengths depend on the layout and on vertex degrees, so we pad every vertex polyomino to a common size T, the size of the largest polyomino, by appending *filler* tiles to it. Filler tiles may lie on any tile except a midpoint, as long as they keep S_v connected; avoiding midpoints ensures that each midpoint stays covered by exactly two polyominoes. All n vertex polyominoes then have the same size T, e.g., $T = 7$ in Fig. 14.

Step 5: Setting the sampling gaps. Finally, we set the maximum tile sampling gaps. We set $\bar{g}^{M}_{i,j} = n$ for all midpoint tiles (i, j), requiring each to be covered by a selected polyomino at least once across the n frames, and set $\bar{g}^{M}_{i,j} = \text{null}$ on all other tiles, imposing no requirement anywhere else. In Fig. 14, the three midpoint tiles (orange) receive gap of $n = 3$.

Step 6: Equivalence to Minimum Vertex Cover. Because the instance contains exactly one polyomino per vertex, selecting polyominoes is the same as selecting vertices in G. Since only the polyominoes S_u and S_v of edge (u, v)'s two endpoints cover $m_{(u,v)}$, satisfying $m_{(u,v)}$'s sampling-gap requirement means selecting at least one of S_u or S_v, so covering all midpoints is equivalent to selecting a vertex cover.

Because all vertex polyominoes have the same size T, minimizing the total number of selected tiles is equivalent to minimizing the number of selected polyominoes, which is equivalent to minimizing the number of selected vertices. The construction is polynomial, so a polynomial-time algorithm for this pruning variant would solve Minimum Vertex Cover, and the variant is therefore NP-hard. Fig. 14 illustrates the reduction on a triangle graph.

A.1.2 Polyomino Pruning Encodes the Relaxed Variant. We now adapt the reduction to polyomino pruning as formulated in §5.2, where every tile carries a sampling gap.

Step 1: Adding a universal polyomino. We call the original n frames, one per vertex, the *vertex frames*, and add one new frame f_0 before the first of them, increasing the number of frames from n to $N = n + 1$ (Fig. 15). We also extend the tile grid of every frame by one extra column on the right, which we call the *anchor* column; the vertex polyominoes lie entirely within the original grid, so no vertex polyomino covers any anchor tile. Frame f_0 holds a single *universal* polyomino U covering the entire extended grid. The anchor column's tiles are thus covered by U alone.

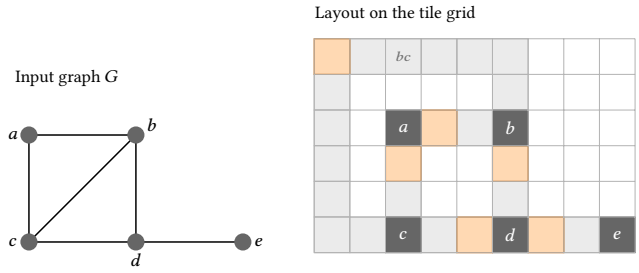


Figure 13: Laying a graph out on the tile grid. Left A planar input graph with maximum degree three. Right Its layout. Each vertex occupies a tile, each edge becomes a path of tiles routed along grid rows and columns, possibly with bends (edge bc is routed around the top-left of the grid, bending at its corners), and distinct edge paths share no tiles. One interior tile of each edge path is marked as the edge's midpoint (orange).

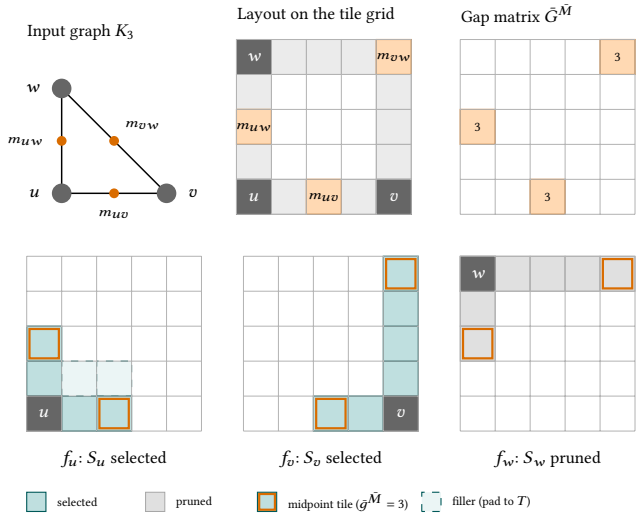


Figure 14: The pruning reduction on the triangle K_3 ($n = 3$, minimum vertex cover size two). Top Left The input graph with each edge's midpoint marked. Top Middle Its layout on the shared tile grid. Top Right The gap matrix $\bar{G}^{\bar{M}}$: every midpoint tile receives gap of $n = 3$, forcing the selection of at least one of its edge's two endpoint polyominoes, while all other tiles carry no sampling gap ($\bar{g}^{\bar{M}}_{i,j} = \text{null}$, blank). Bottom The three frames, one vertex polyomino per frame, each padded to $T = 7$ tiles. Each midpoint is covered by exactly its edge's two endpoint polyominoes, e.g., $m_{(v,w)}$ by S_v and S_w only, so a feasible selection must pick an endpoint polyomino per edge, i.e., a vertex cover. Selecting S_u and S_v (the cover $\{u, v\}$) satisfies all three midpoints at $2T = 14$ tiles, and S_w is pruned; no single polyomino covers all three midpoints, matching K_3's cover size of two.

Step 2: Setting the sampling gaps on every tile. We give every non-midpoint tile, including the anchor column's tiles, gap N, and every midpoint tile gap n. Every tile now carries a sampling gap, as the original pruning problem requires, and no gap exceeds N, the number of frames.

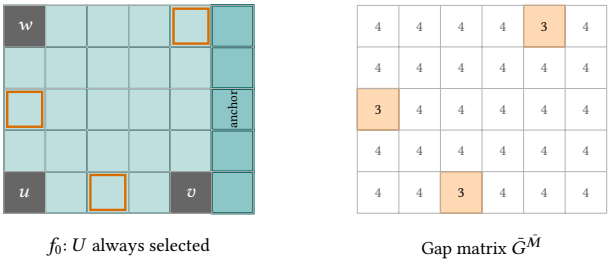


Figure 15: The universal polyomino U from §A.1.2, shown for the triangle example of Fig. 14. Left U occupies its own frame f_0 ($N = n + 1 = 4$), prepended to the vertex frames, and covers the entire grid. Every frame's grid is extended by the anchor column (darker); no vertex polyomino covers its tiles, so they are covered by U alone. Vertex and midpoint tiles are marked for orientation only. Right The gap matrix $\bar{G}^{\bar{M}}$: midpoint tiles (orange) keep their gap of $n = 3$, and every other tile, including the anchor column, receives gap of $N = 4$. The anchor tiles' windows can be satisfied only by U, forcing U into every feasible selection; U then satisfies the windows of all non-midpoint tiles, while each midpoint (outlined) is still forced by its vertex-frames window to select an endpoint polyomino.

Step 3: Equivalence to the relaxed variant. A non-midpoint tile carries gap N, and a video of N frames contains exactly one window of N consecutive frames, namely the full video. The anchor column's tiles are covered only by U, so their windows can be satisfied only by selecting U, and every feasible selection must include U. Selecting U in turn satisfies every non-midpoint tile, since U covers every tile. A midpoint tile $m_{(u,v)}$ of an edge (u, v) carries gap n, and the $N = n + 1$ frames contain exactly two windows of n consecutive frames. One window spans only the n vertex frames; we call it the *vertex-frames window*. It still contains just S_u and S_v covering $m_{(u,v)}$, so it still forces an endpoint polyomino per edge. The other window contains U and is already satisfied.

A feasible selection thus always consists of U plus a set of vertex polyominoes, costing $|U|$ plus T per selected vertex, so the correspondence to vertex covers holds as before, and polyomino pruning is NP-hard even when every tile carries a gap of at most N.

A.2 Polyomino Packing is NP-hard

Two-dimensional rectangular bin packing is a well-known NP-hard problem [9, 12, 13, 21], and our polyomino packing problem strictly generalizes it. Since rectangles are a special case of polyominoes, any instance of 2D rectangular bin packing can be expressed as an instance of polyomino packing. Therefore, polyomino packing is at least as hard as 2D rectangular bin packing.